

Skill Session 3

Task 1

To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality

```
j_rakshitha@RAKSHITHA:~/ossp_project$ nano src/history_shell.c
j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/history_shell
.c -o build/history_shell
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/history_shell
History Shell
Use Up/Down arrows for command history.
Type 'exit' to quit.

myshell> hello
Command entered: hello
myshell> linux
Command entered: linux
myshell> test
Command entered: test
myshell> hello
Command entered: hello
myshell> exit
Exiting shell...
j_rakshitha@RAKSHITHA:~/ossp_project$
```

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <termios.h>
#include <string.h>

#define MAX_INPUT 100
#define MAX_HISTORY 10

char history[MAX_HISTORY][MAX_INPUT];
int history_count = 0;

void enable_raw_mode(struct termios *original) {
    struct termios raw;

    tcgetattr(STDIN_FILENO, original);
    raw = *original;

    raw.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}

void disable_raw_mode(struct termios *original) {
    tcsetattr(STDIN_FILENO, TCSAFLUSH, original);
}

void add_history(const char *command) {
    if (strlen(command) == 0)
        return;

    if (history_count < MAX_HISTORY) {
        strcpy(history[history_count], command);
        history_count++;
    } else {
        for (int i = 1; i < MAX_HISTORY; i++) {
            strcpy(history[i - 1], history[i]);
        }
    }
}
```

```
    }

    strcpy(history[MAX_HISTORY - 1], command);
}

void clear_line(int length) {
    for (int i = 0; i < length; i++) {
        printf("\b \b");
    }

    fflush(stdout);
}

int main() {
    struct termios original;

    char buffer[MAX_INPUT];
    char ch;

    int position;
    int history_index;

    printf("History Shell\n");
    printf("Use Up/Down arrows for command history.\n");
    printf("Type 'exit' to quit.\n\n");

    enable_raw_mode(&original);

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        position = 0;
        history_index = history_count;

        buffer[0] = '\0';
```

```
        break;

    /* Enter key */
    if (ch == '\n' || ch == '\r') {
        buffer[position] = '\0';

        printf("\n");

        break;
    }

    /* Backspace */
    if (ch == 127 || ch == '\b') {

        if (position > 0) {
            position--;

            buffer[position] = '\0';

            printf("\b \b");
            fflush(stdout);
        }

        continue;
    }

    /* Escape sequence */
    if (ch == 27) {

        char seq1, seq2;

        if (read(STDIN_FILENO, &seq1, 1) <= 0)
            continue;

        if (read(STDIN_FILENO, &seq2, 1) <= 0)
            continue;
```

```
/* Up arrow: ESC [ A */
if (seq1 == '[' && seq2 == 'A') {

    if (history_count > 0 && history_index > 0) {

        clear_line(position);

        history_index--;

        strcpy(buffer, history[history_index]);

        position = strlen(buffer);

        printf("%s", buffer);

        fflush(stdout);

    }
}

/* Down arrow: ESC [ B */
else if (seq1 == '[' && seq2 == 'B') {

    if (history_index < history_count - 1) {

        clear_line(position);

        history_index++;

        strcpy(buffer, history[history_index]);

        position = strlen(buffer);

        printf("%s", buffer);

        fflush(stdout);

    }
}
else if (history_index == history_count - 1) {
```

```
        continue;
    }

    /* Normal character */
    if (position < MAX_INPUT - 1) {
        buffer[position++] = ch;
        buffer[position] = '\0';
        printf("%c", ch);
        fflush(stdout);
    }
}

/* Exit */
if (strcmp(buffer, "exit") == 0) {
    printf("Exiting shell...\n");
    break;
}

/* Store command */
if (position > 0) {
    add_history(buffer);
    printf("Command entered: %s\n", buffer);
}
}

disable_raw_mode(&original);

return 0;
}
```

Task 2

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind

```
j_rakshitha@RAKSHITHA:~/ossp_project$ nano src/memory_demo.c

j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/memory_demo.c -o build/memory_demo
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/memory_demo
Dynamic Memory Demo
Enter a command: hello

Command stored: hello
Final buffer size: 10 bytes
Command added to linked list.

Linked List Contents:
- hello

All allocated memory has been released.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/memory_demo
Dynamic Memory Demo
Enter a command: this is a very long command to test dynamic memory allocation

Buffer resized to 20 bytes.

Buffer resized to 40 bytes.

Buffer resized to 80 bytes.

Command stored: this is a very long command to test dynamic memory allocation
Final buffer size: 80 bytes
Command added to linked list.

Linked List Contents:
- this is a very long command to test dynamic memory allocation

All allocated memory has been released.
```

```
j_rakshitha@RAKSHITHA:~/ossp_project$ valgrind --version
valgrind-3.26.0
j_rakshitha@RAKSHITHA:~/ossp_project$ valgrind --leak-check=full ./build/memory_demo
==2805== Memcheck, a memory error detector
==2805== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2805== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2805== Command: ./build/memory_demo
==2805==
Dynamic Memory Demo
Enter a command: hello

Command stored: hello
Final buffer size: 10 bytes
Command added to linked list.

Linked List Contents:
- hello

All allocated memory has been released.
==2805==
==2805== HEAP SUMMARY:
==2805==    in use at exit: 0 bytes in 0 blocks
==2805==   total heap usage: 5 allocs, 5 frees, 2,080 bytes allocated
==2805==
==2805== All heap blocks were freed -- no leaks are possible
==2805==
==2805== For lists of detected and suppressed errors, rerun with: -s
==2805== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
j_rakshitha@RAKSHITHA:~/ossp_project$ |
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    char *command;
    struct Node *next;
} Node;

int main() {
    char *buffer;
    size_t size = 10;
    size_t length = 0;

    Node *head = NULL;
    Node *tail = NULL;

    buffer = malloc(size);

    if (buffer == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Dynamic Memory Demo\n");
    printf("Enter a command: ");

    int ch;

    while ((ch = getchar()) != '\n' && ch != EOF) {

        if (length + 1 >= size) {
            size *= 2;

            char *temp = realloc(buffer, size);

            if (temp == NULL) {
```

```
    char *temp = realloc(buffer, size);

    if (temp == NULL) {
        printf("\nMemory reallocation failed.\n");
        free(buffer);
        return 1;
    }

    buffer = temp;

    printf("\nBuffer resized to %zu bytes.\n", size);
}

buffer[length++] = (char)ch;

buffer[length] = '\0';

printf("\nCommand stored: %s\n", buffer);
printf("Final buffer size: %zu bytes\n", size);

Node *new_node = malloc(sizeof(Node));

if (new_node == NULL) {
    free(buffer);
    return 1;
}

new_node->command = malloc(strlen(buffer) + 1);

if (new_node->command == NULL) {
    free(new_node);
    free(buffer);
    return 1;
}
```

```

    if (new_node->command == NULL) {
        free(new_node);
        free(buffer);
        return 1;
    }

    strcpy(new_node->command, buffer);
    new_node->next = NULL;

    if (head == NULL) {
        head = new_node;
        tail = new_node;
    } else {
        tail->next = new_node;
        tail = new_node;
    }

    printf("Command added to linked list.\n");

    printf("\nLinked List Contents:\n");

    Node *current = head;

    while (current != NULL) {
        printf("- %s\n", current->command);
        current = current->next;
    }

    current = head;

    while (current != NULL) {
        Node *temp = current;
        current = current->next;

        free(temp->command);
        free(temp);

```

```

        free(temp->command);
        free(temp);
    }

    free(buffer);

    printf("\nAll allocated memory has been released.\n");

    return 0;
}
|

```

Session 4

Task 1

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output

```
j_rakshitha@RAKSHITHA:~/ossp_project$ nano src/tokenizer.c
j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/tokenizer.c -o build/tokenizer
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/tokenizer
Tokenization Demo
Enter a command: ls -l

Tokens Generated:
Token 1: [ls]
Token 2: [-l]

Total Tokens: 2

Tokenization completed successfully.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/tokenizer
Tokenization Demo
Enter a command: Tokens Generated:
Token 1: [echo]
Token 2: [hell
Tokens Generated:
Token 1: [Tokens]
Token 2: [Generated:]

Total Tokens: 2

Tokenization completed successfully.
o]
Token 3: [world]
```

```
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/tokenizer
```

```
Tokenization Demo
```

```
Enter a command: cat file.txt | grep hello
```

```
Tokens Generated:
```

```
Token 1: [cat]
```

```
Token 2: [file.txt]
```

```
Token 3: [|]
```

```
Token 4: [grep]
```

```
Token 5: [hello]
```

```
Total Tokens: 5
```

```
Tokenization completed successfully.
```

```
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/tokenizer
```

```
Tokenization Demo
```

```
Enter a command: echo hello world
```

```
Tokens Generated:
```

```
Token 1: [echo]
```

```
Token 2: [hello]
```

```
Token 3: [world]
```

```
Total Tokens: 3
```

```
Tokenization completed successfully.
```

```
j_rakshitha@RAKSHITHA:~/ossp_project$ |
```

```
GNU nano 8.7.1 src/tokenizer.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 20
#define MAX_TOKEN_LENGTH 50

typedef struct {
    char value[MAX_TOKEN_LENGTH];
} Token;

int tokenize(char *input, Token tokens[]) {
    int count = 0;
    int i = 0;

    while (input[i] != '\0' && count < MAX_TOKENS) {

        /* Skip whitespace */
        while (isspace((unsigned char)input[i])) {
            i++;
        }

        if (input[i] == '\0') {
            break;
        }

        /* Identify delimiters */
        if (input[i] == '|' ||
            input[i] == '<' ||
            input[i] == '>' ||
            input[i] == '&') {

            tokens[count].value[0] = input[i];
            tokens[count].value[1] = '\0';

            count++;
            i++;

            continue;
        }
    }
}
```

```
        if (j < MAX_TOKEN_LENGTH - 1) {
            tokens[count].value[j++] = input[i];
        }

        i++;
    }

    tokens[count].value[j] = '\0';
    count++;
}

return count;
}

int main() {
    char input[200];
    Token tokens[MAX_TOKENS];

    printf("Tokenization Demo\n");
    printf("Enter a command: ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        return 1;
    }

    int token_count = tokenize(input, tokens);

    printf("\nTokens Generated:\n");

    for (int i = 0; i < token_count; i++) {
        printf("Token %d: [%s]\n", i + 1, tokens[i].value);
    }

    printf("\nTotal Tokens: %d\n", token_count);

    printf("\nTokenization completed successfully.\n");

    return 0;
}
```

Task 2

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

```
j_rakshitha@RAKSHITHA:~/ossp_project$ nano src/parser.c
j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/parser.c -o build/parser
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/parser
Parser Demo
Enter a command: ls -l

Tokens received: 2
Token 1: [ls]
Token 2: [-l]

Validating syntax...
Syntax is valid.

Parse Tree:
Command: ls
Arguments:
└─ -l

Parser completed successfully.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/parser
Parser Demo
Enter a command: echo hello world

Tokens received: 3
Token 1: [echo]
Token 2: [hello]
Token 3: [world]

Validating syntax...
Syntax is valid.

Parse Tree:
Command: echo
Arguments:
└─ hello
   └─ world

Parser completed successfully.
```

```
Parser completed successfully.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/parser
Parser Demo
Enter a command: | ls

Tokens received: 2
Token 1: [|]
Token 2: [ls]

Validating syntax...
Error: Command cannot start with '|'.
Parsing failed.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/parser
Parser Demo
Enter a command: ls |

Tokens received: 2
Token 1: [ls]
Token 2: [|]

Validating syntax...
Error: Command cannot end with '|'.
Parsing failed.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/parser
Parser Demo
Enter a command:

Tokens received: 0

Validating syntax...
Error: Empty command.
Parsing failed.
j_rakshitha@RAKSHITHA:~/ossp_project$ |
```

```

GNU nano 8.7.1 src/parser.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_TOKENS 20
#define MAX_TOKEN_LENGTH 50

typedef struct {
    char value[MAX_TOKEN_LENGTH];
} Token;

typedef struct {
    char command[MAX_TOKEN_LENGTH];
    char arguments[MAX_TOKENS][MAX_TOKEN_LENGTH];
    int argument_count;
} Command;

int validate_tokens(Token tokens[], int count) {
    if (count == 0) {
        printf("Error: Empty command.\n");
        return 0;
    }

    if (strcmp(tokens[0].value, "|") == 0 ||
        strcmp(tokens[0].value, ">") == 0 ||
        strcmp(tokens[0].value, "<") == 0) {
        printf("Error: Command cannot start with '%s'.\n",
            tokens[0].value);
        return 0;
    }

    for (int i = 0; i < count - 1; i++) {
        if (strcmp(tokens[i].value, "|") == 0 &&
            strcmp(tokens[i + 1].value, "|") == 0) {
            printf("Error: Consecutive pipe operators.\n");
            return 0;
        }
    }

    if (strcmp(tokens[count - 1].value, "|") == 0 ||
        strcmp(tokens[count - 1].value, ">") == 0 ||
        strcmp(tokens[count - 1].value, "<") == 0) {
        printf("Error: Command cannot end with '%s'.\n",
            tokens[count - 1].value);
        return 0;
    }

    return 1;
}

void parse_command(Token tokens[], int count, Command *command) {

```

```
}|  
  
void parse_command(Token tokens[], int count, Command *command) {  
    strcpy(command->command, tokens[0].value);  
    command->argument_count = 0;  
  
    for (int i = 1; i < count; i++) {  
        if (strcmp(tokens[i].value, "|") == 0 ||  
            strcmp(tokens[i].value, ">") == 0 ||  
            strcmp(tokens[i].value, "<") == 0) {  
            continue;  
        }  
  
        if (command->argument_count < MAX_TOKENS) {  
            strcpy(  
                command->arguments[command->argument_count],  
                tokens[i].value  
            );  
  
            command->argument_count++;  
        }  
    }  
}  
  
void print_parse_tree(Command *command) {  
    printf("\nParse Tree:\n");  
    printf("Command: %s\n", command->command);  
  
    if (command->argument_count == 0) {  
        printf("Arguments: None\n");  
    } else {  
        printf("Arguments:\n");  
  
        for (int i = 0; i < command->argument_count; i++) {  
            printf("    └─ %s\n",  
                command->arguments[i]);  
        }  
    }  
}  
  
int main() {  
    char input[200];  
    Token tokens[MAX_TOKENS];  
    Command command;  
  
    printf("Parser Demo\n");  
    printf("Enter a command: ");  
  
    if (fgets(input, sizeof(input), stdin) == NULL) {  
        return 1;  
    }  
}
```

```

int main() {
    char input[200];
    Token tokens[MAX_TOKENS];
    Command command;

    printf("Parser Demo\n");
    printf("Enter a command: ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    int count = 0;

    char *token = strtok(input, " \t");

    while (token != NULL && count < MAX_TOKENS) {
        strcpy(tokens[count].value, token);
        count++;

        token = strtok(NULL, " \t");
    }

    printf("\nTokens received: %d\n", count);

    for (int i = 0; i < count; i++) {
        printf("Token %d: [%s]\n",
            i + 1,
            tokens[i].value);
    }

    printf("\nValidating syntax...\n");

    if (!validate_tokens(tokens, count)) {
        printf("Parsing failed.\n");
        return 1;
    }

    printf("Syntax is valid.\n");

    parse_command(tokens, count, &command);

    print_parse_tree(&command);

    printf("\nParser completed successfully.\n");

    return 0;
}

```

Practice Task

To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.

```
Setting up tree (2.3.1-1) ...
Processing triggers for man-db (2.13.1-1bu
j_rakshitha@RAKSHITHA:~/OSSP$ tree
.
├── LICENSE
├── Makefile
├── README.md
├── assets
│   └── architecture.png
├── bin
│   └── my_shell
├── docs
│   ├── design.md
│   └── report.pdf
├── include
│   ├── builtin.h
│   ├── executor.h
│   ├── parser.h
│   └── shell.h
├── obj
├── scripts
│   └── run.sh
├── src
│   ├── builtin.c
│   ├── executor.c
│   ├── main.c
│   ├── parser.c
│   └── shell.c
└── tests
    ├── test_executor.c
    └── test_parser.c

9 directories, 19 files
j_rakshitha@RAKSHITHA:~/OSSP$ |
```

To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing.

```
j_rakshitha@RAKSHITHA:~$ echo $$
374
j_rakshitha@RAKSHITHA:~$ ps
  PID TTY          TIME CMD
  374 pts/0    00:00:00 bash
 2431 pts/0    00:00:00 ps
j_rakshitha@RAKSHITHA:~$ pstree
systemd--agetty
          |
          +--chronyd--starter--chronyd--chronyd
          |
          +--cron
          |
          +--dbus-daemon
          |
          +--init-systemd(ub)
              |
              +--SessionLeader--Relay(374)--bash--pstree
              |
              +--init--{init}
              |
              +--login--bash
              |
              +--{init-systemd(ub)}
          |
          +--networkd-dispatcher
          |
          +--packagekitd--3*[packagekitd]
          |
          +--polkitd--3*[polkitd]
          |
          +--rsyslogd--3*[rsyslogd]
          |
          +--systemd--(sd-pam)
          |
          +--systemd-journald
          |
          +--systemd-logind
          |
          +--systemd-resolved
          |
          +--systemd-udevd
          |
          +--unattended-upgr--{unattended-upgr}
          |
          +--wsl-pro-service--7*[wsl-pro-service]
          |
          +--[[200-echo $$-
j_rakshitha@RAKSHITHA:~$ echo $$
374
j_rakshitha@RAKSHITHA:~$ ps -ef
UID          PID    PPID  C   TIME TTY          TIME CMD
root           1         0  0  04:46 ?        00:00:01 /usr/lib/systemd/systemd --system --deserialize=20
root           2         1  0  04:46 hvc0     00:00:00 /init
root           7         2  0  04:46 hvc0     00:00:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
root          47         1  0  04:46 ?        00:00:00 /usr/lib/systemd/systemd-journald
systemd+     82         1  0  04:46 ?        00:00:00 /usr/lib/systemd/systemd-resolved
root         80         1  0  04:46 ?        00:00:01 /usr/lib/systemd/systemd-udevd
root        108         1  0  04:46 ?        00:00:00 /bin/sh /usr/lib/systemd/scripts/chronyd-starter.sh -n -f 1
root        116         1  0  04:46 ?        00:00:00 /usr/sbin/cron -f -P
message+    121         1  0  04:46 ?        00:00:00 @dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
root        156         1  0  04:46 ?        00:00:00 /usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-triggers
root        162         1  0  04:46 ?        00:00:00 /usr/lib/systemd/systemd-logind
syslog      187         1  0  04:46 ?        00:00:00 /usr/sbin/rsyslogd -n -iNONE
root        211         1  0  04:46 ttyl     00:00:00 /usr/sbin/agetty --noret --noclear --issue-file=/etc/issue:/etc/issue.d/run/issue.d:/usr/lib/issue.d - linux
_chrony     232        108  0  04:46 ?        00:00:00 /usr/sbin/chronyd -n -f 1 -x
_chrony     252        232  0  04:46 ?        00:00:00 /usr/sbin/chronyd -n -f 1 -x
root        276         1  0  04:46 ?        00:00:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
root        367         2  0  04:46 ?        00:00:00 /init
root        368        367  0  04:46 ?        00:00:00 /init
j_raksh+    374        368  0  04:46 pts/0    00:00:00 -bash
root        375         2  0  04:46 ?        00:00:00 login -- j_rakshitha
j_raksh+    421         1  0  04:46 ?        00:00:00 /usr/lib/systemd/systemd --user --deserialize=12
j_raksh+    423        421  0  04:46 ?        00:00:00 (sd-pam)
j_raksh+    451        375  0  04:46 pts/1    00:00:00 -bash
root       1088         1  0  05:49 ?        00:00:00 /usr/libexec/wsl-pro-service
polkitd    1190         1  0  05:49 ?        00:00:00 /usr/lib/polkit-1/polkitd --no-debug --log-level=notice
j_raksh+   2409        374  0  05:56 pts/0    00:00:00 ps -ef
j_rakshitha@RAKSHITHA:~$ pwd
/home/j_rakshitha
j_rakshitha@RAKSHITHA:~$ date
Mon Aug 3 05:57:01 UTC 2026
j_rakshitha@RAKSHITHA:~$ whoami
j_rakshitha
j_rakshitha@RAKSHITHA:~$ |
```